

Computer Architectures

Exam of 25/06/2026

B2

Please read the following notes before proceeding

- 1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
- 2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
- 3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
; to do: add code
MOV r0, #10      ; example of a return value
BX LR
ENDP
```

- 4) To earn points on all exercises, it is recommended that you write at least some code for each.
- 5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets []
Left bracket: ATL GR + [
Right bracket: ALT GR +]
- Curly brackets { }
Left bracket: LSHIFT + ALT GR + [
Right bracket: LSHIFT + ALT GR +]
- Hash #: ALT GR + à

Bitwise operations in C

Left shift by N bits = $A \ll N$
Right shift by N bits = $A \gg N$

A XOR B = $A \wedge B$
A AND B = $A \& B$ (&& is the logical AND)
A OR B = $A | B$ (|| is the logical OR)

Question 1 (10 points)

In the *Mastermind* game, a player tries to discover a secret code by trial and error.
In our implementation, the secret code consists of 4 digits. Each digit is in the range [0-3].

Write the *Mastermind* subroutine in ARM assembly language. You must follow this prototype:

int Mastermind(int guess[4], int secret[4], int usedGuess [4], int usedSecret [4]);

The four parameters are addresses of arrays containing four integers.

In the *Reset_Handler*, before calling the subroutine, you can initialize the elements of *guess* (the player's try) and *secret* (the code to be discovered) with values of your choice. Instead, all the elements of *usedGuess* and *usedSecret* must be initialized to zero.

The *Mastermind* subroutine computes:

- the exact matches: the number of digits that have the same value and appear in the same position in both arrays (*guess* and *secret*);
- the partial matches: the number of digits in *guess* that are not exact matches, but appear in a different position in *secret*.

Example: *guess* = [0, 1, 2, 3]; *secret* = [1, 2, 2, 0].

There is one exact match (digit 2 in the third position) and two partial matches (digits 0 and 1).

The subroutine implements the following pseudocode:

```
exact = 0;
partial = 0;
for (i = 0; i < 4; i ++) {
    if (guess[i] == secret[i]) {
        exact ++;
        usedGuess[i] = 1;    /* mark the digit as used */
        usedSecret[i] = 1;
    }
}
for (i = 0; i < 4; i ++)
    for (j = 0; j < 4; j ++)
        if (usedGuess[i] == 0 && usedSecret[j] == 0 &&
            guess[i] == secret[j]) {
            partial ++;
            usedGuess[i] = 1;
            usedSecret[j] = 1;
            break;    /* ends the inner loop */
        }
return (2exact - 1) << 4 + (2partial - 1)
```

In this pseudocode, the arrays *usedGuess* and *usedSecret* are used to mark digits that have already been matched. A value of 0 indicates that the digit has not been matched yet, whereas a value of 1 indicates that it has already been matched.

In the example above, at the end of the procedure we have:

usedGuess = [1, 1, 1, 0]

usedSecret = [1, 0, 1, 1]

exact = 1

$partial = 2$

$(2^{exact} - 1) = 1$

$(2^{exact} - 1) \ll 4 = 16$

$(2^{partial} - 1) = 3$

The subroutine returns 19.

Question 2 (8 points)

In this part, you implement a game of *Mastermind* with the LPC1768 board.

In the `main` function, start a timer of your choice.

The player presses the joystick (i.e., *select* function) to start the game. In this first moment:

- all LEDs are turned off
- the player's guess is initialized to 0000
- the timer counter is used to generate the secret code.

In detail, the secret code is generated from the four least significant hexadecimal digits of the timer counter. Each hexadecimal digit is masked with 0x3 in order to obtain a value in the range [0-3].

Example: `timerCounter = 0x00121AB6`

`secretCode[0] = 0x6 AND 0x3 = 01102 AND 00112 = 00102 = 0x2`

`secretCode[1] = 0xB AND 0x3 = 10112 AND 00112 = 00112 = 0x3`

`secretCode[2] = 0xA AND 0x3 = 10102 AND 00112 = 10102 = 0x2`

`secretCode[3] = 0x1 AND 0x3 = 00012 AND 00112 = 00012 = 0x1`

The player moves the joystick to choose the four digits of their guess:

- when the joystick moves up, the first digit of the guess is incremented. If the value becomes 4, then it is reset to 0. The binary representation of `guess[0]` is displayed on LEDs 10 and 11. For example, if `guess[0] = 2`, then LED 10 is turned on and LED 11 is turned off.
- when the joystick moves right, the second digit of the guess is incremented. If the value becomes 4, then it is reset to 0. The binary representation of `guess[1]` is displayed on LEDs 8 and 9.
- when the joystick moves down, the third digit of the guess is incremented. If the value becomes 4, then it is reset to 0. The binary representation of `guess[2]` is displayed on LEDs 6 and 7.
- when the joystick moves left, the fourth digit of the guess is incremented. If the value becomes 4, then it is reset to 0. The binary representation of `guess[3]` is displayed on LEDs 4 and 5.

When the player is satisfied with their guess, they press the joystick. The `Mastermind` subroutine is then called passing `guess`, `secret` and two arrays initialized to zero. The value returned by the subroutine is shown on the LEDs, assuming LED4 represents the most significant digit and LED 11 the least significant one.

The player can press the joystick again to compose another guess. At this moment:

- all LEDs are turned off
- the player's guess is initialized to 0000

Note that the secret code must not be changed.

Then, the game continues alternating between joystick movements to set the guess and subroutine calls to compute the number of exact and partial matches. The game ends when the number of exact matches is 4, i.e., LEDs 4-7 are turned on and LEDs 8-11 are turned off. No specific action is required when the game ends.